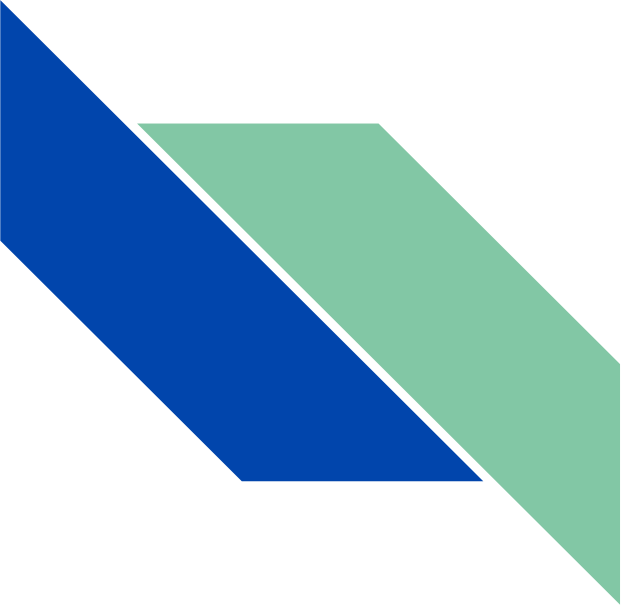


PULS events

Puls-Events RAG

Développement d’un assistant pour la recommandation d'évènements culturels

Rafael CEREZO MARTIN

Mars 2026

Introducción

Objectifs du projet

Entreprise cliente : Puls-Events — Plateforme de recommandations culturelles personnalisées

Problématique : Comment permettre aux utilisateurs d'interroger en langage naturel une base d'événements culturels dynamique, avec des réponses contextualisées, précises et sourcées.

Objectif principal : Concevoir et déployer un POC de système RAG *(Retrieval-Augmented Generation)* combinant LangChain, Mistral et FAISS, exposé via une API REST avec architecture hexagonale.

Stack Technique (RAG)

- **API Open Agenda :** Collecte des événements culturels avec filtres géographiques et temporels. **Pagination automatique**.
- **PostgreSQL & FAISS :** Stockage structuré des événements et index vectoriel sémantique (1024 dim). **Architecture hexagonale** IVectorStore / IEventRepository.
- **Pipeline RAG LangChain :** Vectorisation Mistral Embed, recherche ANN Top-K, génération augmentée Mistral LLM avec guardrails.

Exigences Techniques & Métier

- **Architecture Hexagonale :** Ports & Adapters découplés. Swap FAISS → Pinecone ou Mistral → OpenAI sans modifier le domaine métier.
- **API REST FastAPI :** Endpoints /ask · /rebuild · /health · /metadata. Swagger UI auto-généré. Gestion des erreurs robuste.
- **Évaluation RAGAS :** Jeu de test annoté, métriques Faithfulness >0.85, Context Recall >0.80. Pipeline CI automatisé.

Introducción

Analyse et Modélisation

Sélection du Corpus

• Open Agenda

Étude de la donnée

• Documentation API

• Response JSON

• Schéma relationnel

Analyse et Modélisation

De l'API OpenAgenda à la structure SQL

Étude de la donnée :
Analyse de la documentation et des réponses JSON d'OpenAgenda pour isoler les champs essentiels (titres, dates, lieux, descriptions).

Schéma relationnel :
Conception de la table events dans PostgreSQL, incluant des champs techniques comme **is_vectorized** et **faiss_id** pour assurer la traçabilité avec l'index vectoriel.



Sélection du Corpus

- **Source choisie :** Utilisation des données réelles de l'agenda du **Jass Club Paris** via l'API OpenAgenda comme échantillon représentatif.
- **Objectif :** Transformer un flux d'événements (concerts, sessions jazz) en un corpus structuré pour tester la précision des réponses du système RAG.

Documentation API

Code	Description
uid	Identifiant unique OpenAgenda de l'événement.
slug	Identifiant texte unique pour l'URL de l'événement.
extIds	Liste de clés/paires identifiant l'événement sur des jeux de données externes.
title	Le titre de l'événement.
description	La description courte de l'événement.
longDescription	La description longue de l'événement.
conditions	Description libre des conditions de participation.
keywords	Liste de mots clés.
image	Illustration principale.
imageCredits	Crédits liés à l'illustration.
registration	Liste des moyens d'inscription : téléphones, email ou liens.
accessibility	Types de handicaps pour lesquels des aménagements sont prévus.
timings	Liste de plages horaires.
age	Créneau min/max d'âge des participants ciblés.
attendanceMode	Mode de participation à l'événement (physique vs en ligne).
locationUid	Identifiant unique OpenAgenda du lieu associé.
onlineAccessLink	Lien hypertexte d'accès à un événement en ligne ou mixte.
links	Liste de paires lien / codes enrichis.
timezone	Fuseau horaire de référence.
status	État de l'événement (ex: Programmé, Reporté, Annulé).
state	Statut dans l'agenda (ex: Publié, Prêt, À modérer).
createdAt	Instant de création de l'événement.
updatedAt	Instant de la dernière mise à jour.

Response JSON


```
{
  "total": 1250,
  "events": [
    {
      "uid": 123456,
      "slug": "concert-jazz-paris-2024",
      "title": {
        "fr": "Concert Jazz au Sunset",
        "en": "Jazz Concert at Sunset"
      },
      "description": {
        "fr": "Une soirée jazz exceptionnelle...",
        "en": "An exceptional jazz evening..."
      },
      "longDescription": {
        "fr": "Description longue détaillée...",
      },
      "conditions": {
        "fr": "Entrée libre / Payant 15€"
      },
      "keywords": {
        "fr": ["jazz", "musique", "concert", "live"]
      },
      "location": {
        "uid": 789,
        "name": "Le Sunset Sunside",
        "address": "60 Rue des Lombards",
        "city": "Paris",
        "postalCode": "75001",
        "department": "Paris",
        "region": "Île-de-France",
        "latitude": 48.8603,
        "longitude": 2.3477,
        "countryCode": "FR"
      },
      "timings": [
        {
          "start": "2024-11-15T20:00:00+01:00",
          "end":   "2024-11-15T23:00:00+01:00"
        }
      ],
      "thumbnail": "https://cdn.openagenda.com/.../image.jpg",
      "links": [
        { "link": "https://www.sunset-sunside.com/..." }
      ],
      "registration": [
        { "value": "https://tickets.example.com/..." }
      ],
      "status": 1,
      "accessibility": {
        "fr": false, "en": false, "es": false, "de": false, "it": false, "pt": false, "nl": false, "ru": false, "zh": false, "ja": false, "ar": false, "pl": false, "vi": false
      },
      "createdAt": "2024-10-01T10:00:00Z",
      "updatedAt": "2024-10-15T14:30:00Z",
      "firstTiming": { "start": "...", "end": "..." },
      "lastTiming":  { "start": "...", "end": "..." }
    }
  ]
}
```


Postgres Table Events



Introducción

Architecture du système RAG

Pipelines d'ingestion et de requête

Architecture globale du système RAG Puls-Events

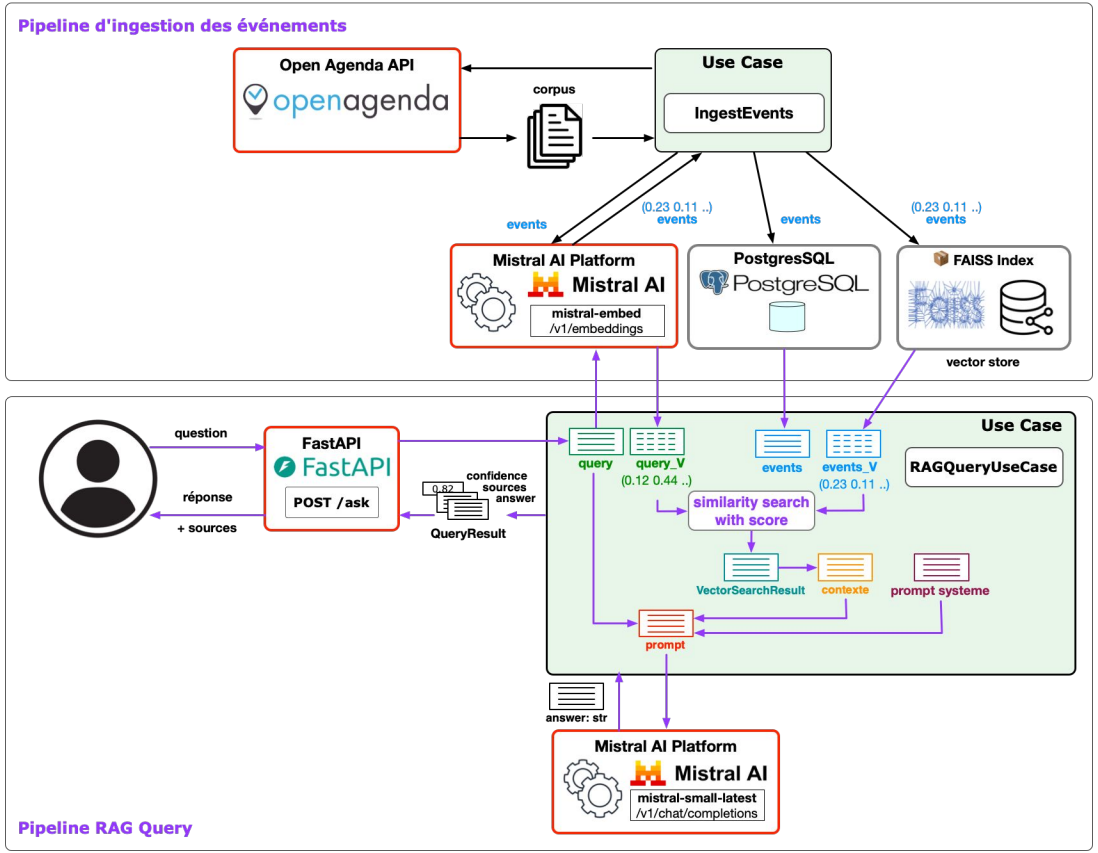
Structurée autour de deux axes majeurs

L'ingestion

Synchronise les événements d'OpenAgenda vers une base vectorielle FAISS via Mistral AI.

Le **pipeline de requête**

Traite les questions via FastAPI, utilisant la recherche de similarité pour générer des réponses augmentées par le contexte

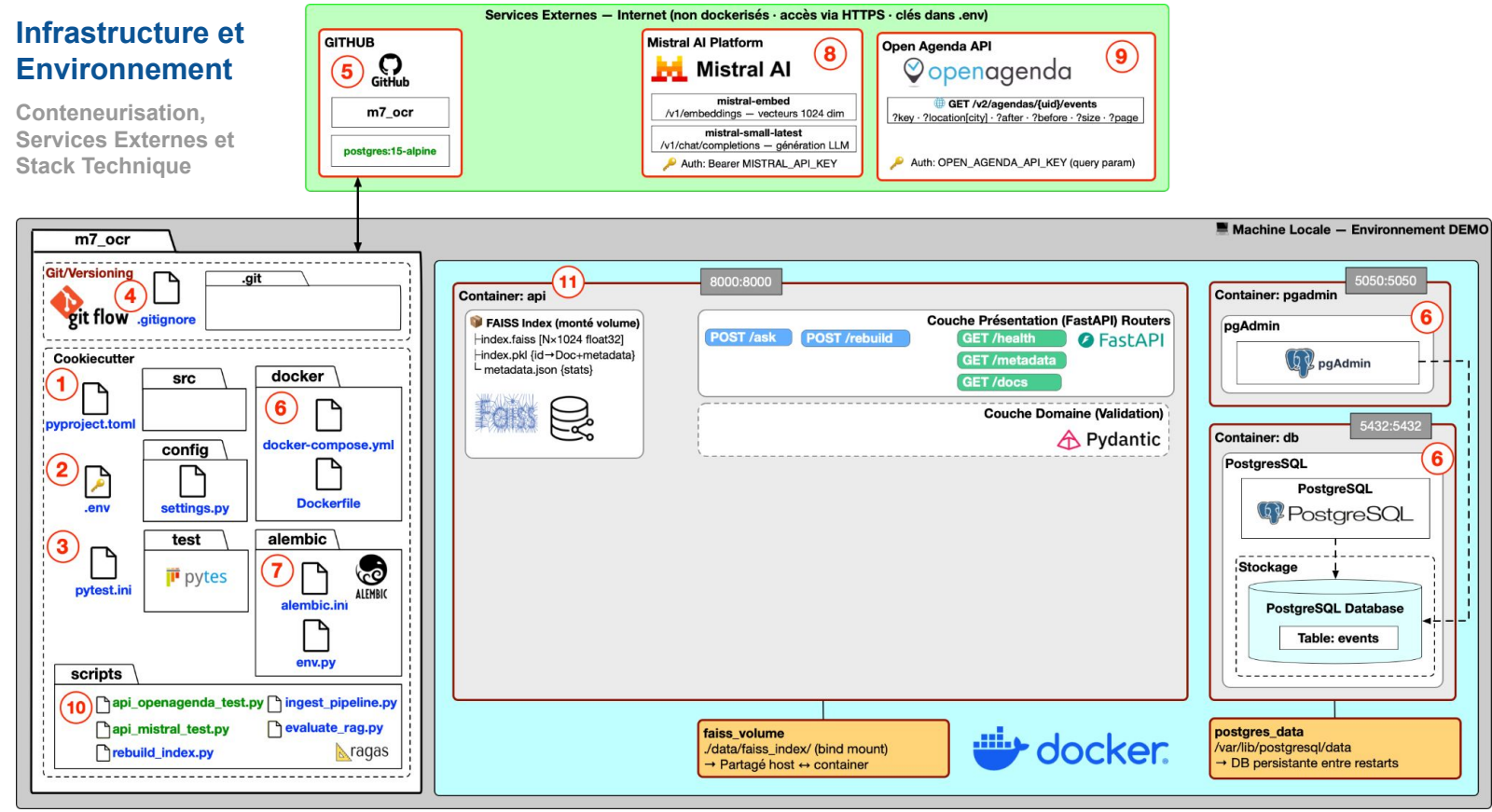


1

Mise en place de l'environnement

Infrastructure et Environnement

Conteneurisation, Services Externes et Stack Technique



Mise en place de l'environnement

1 à 3 : Configuration Initiale : Avec pyproject.toml, gestion des variables d'environnement (.env) et framework de tests pytest.

4 : Versioning : Gestion du cycle de vie du code via Git Flow

5 : CI/CD : Intégration continue avec GitHub

6 & 7 : Orchestration & Base de données : Déploiement multi-conteneurs via Docker Compose et gestion des migrations SQL avec Alembic.

8 & 9: Services Externes : Connexion sécurisée aux API de Mistral AI et OpenAgenda

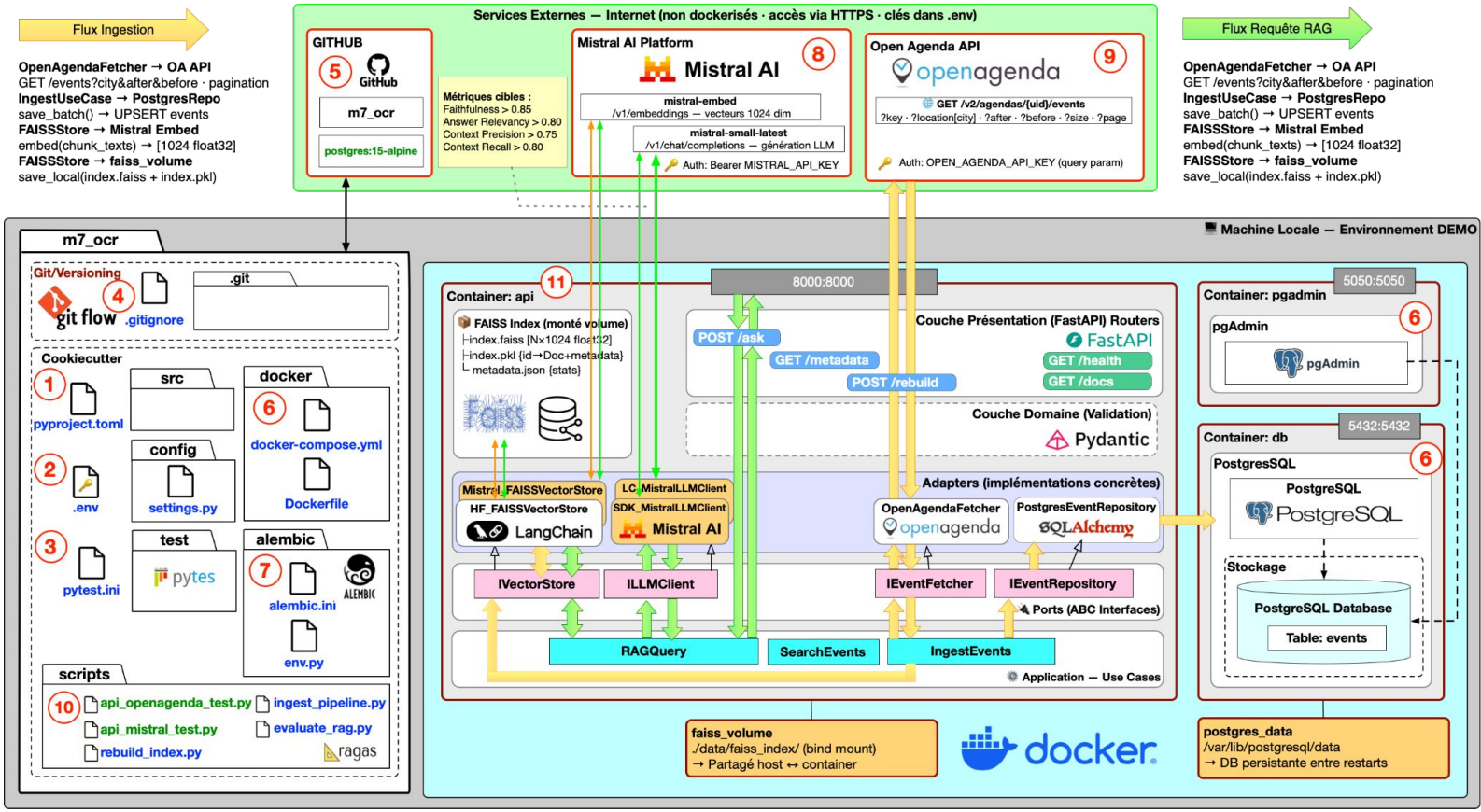
10 : Scripts Utilitaires : Boîte à outils pour les tests

11 : Noyau Applicatif : API FastAPI orchestrant l'index FAISS, la validation Pydantic et la logique métier RAG.

1

Mise en place de l'environnement

Mise en place de l'environnement
1 à 3 : Configuration Initiale : Avec pyproject.toml, gestion des variables d'environnement (.env) et framework de tests pytest.
4 : Versioning : Gestion du cycle de vie du code via Git Flow
5 : CI/CD : Intégration continue avec GitHub
6 & 7 : Orchestration & Base de données : Déploiement multi-conteneurs via Docker Compose et gestion des migrations SQL avec Alembic.
8 & 9: Services Externes : Connexion sécurisée aux API de Mistral AI et OpenAgenda
10 : Scripts Utilitaires : Boîte à outils pour les tests
11 : Noyau Applicatif : API FastAPI orchestrant l'index FAISS, la validation Pydantic et la logique métier RAG.



2

Pipeline d'ingestion des événements

FLux Ingestion

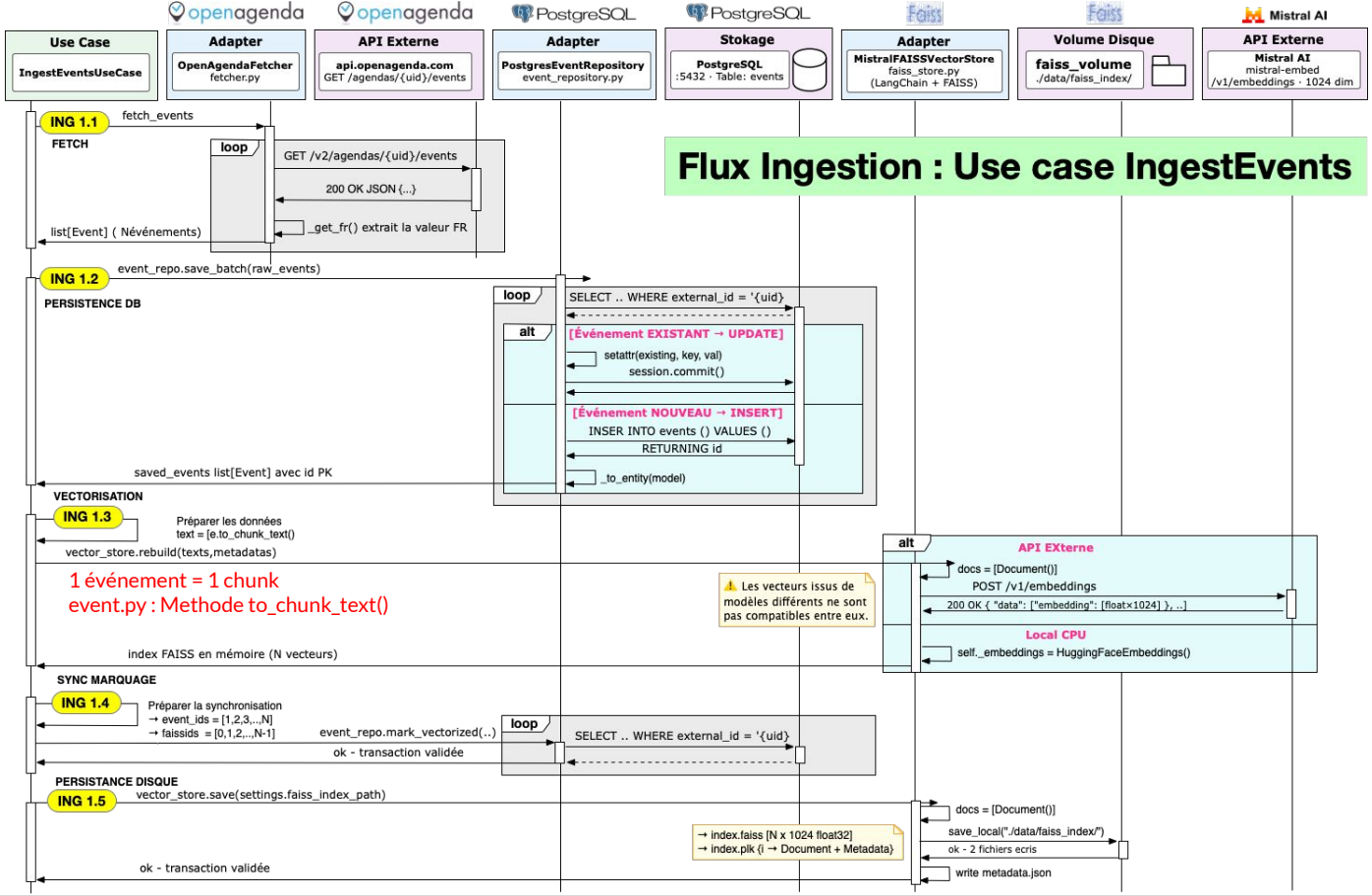
1. FETCH
(IEventFetcher →OpenAgendaFetcher)

2. PERSISTENCE DB
(IEventRepository → PostgresEventRepository)

3. VECTORISATION
(IVectorStore → FAISSVectorStore + Mistral Embed)

4. SYNC MARQUAGE
(IEventRepository)

5. PERSISTANCE DISQUE
(IVectorStore)



2

Pipeline d'ingestion des événements

FLux Ingestion

1. FETCH
(IEventFetcher →OpenAgendaFetcher)

2. PERSISTENCE DB
(IEventRepository → PostgresEventRepository)

3. VECTORISATION
(IVectorStore → FAISSVectorStore + Mistral Embed)

4. SYNC MARQUAGE
(IEventRepository)

5. PERSISTANCE DISQUE
(IVectorStore)

Vérifications des enregistrements.

uv run python .\scripts\read_db.py

PostgreSQL Database

ID	Titre	Ville	Lieu	Début	Sessions	Vecteur	FAISS ID
1	Alexandre Herer 'Oxyd'	PARIS	JASS CLUB PARIS	2026-03-13 00:00	19:30, 21:30, 22:30	✔ oui	0
2	Idrîs-Félix Bahri / Hommage à Ro	PARIS	JASS CLUB PARIS	2026-03-14 00:00	19:30, 21:30, 22:30	✔ oui	1
3	Neil Saidi & Yûr Yesli Quartet /	PARIS	JASS CLUB PARIS	2026-03-15 00:00	19:30	✔ oui	2
4	St Patrick / Entrée Libre ! Jam	PARIS	JASS CLUB PARIS	2026-03-17 00:00	19:00	✔ oui	3
5	ZAMAKAN	PARIS	JASS CLUB PARIS	2026-03-18 00:00	19:30, 21:30	✔ oui	4
6	Azilis	PARIS	JASS CLUB PARIS	2026-03-19 00:00	19:30, 21:30	✔ oui	5
7	Virginie Daïdé Quartet 'While we	PARIS	JASS CLUB PARIS	2026-03-20 00:00	19:30, 21:30, 22:30	✔ oui	6
8	Paris Tokyo Exchange feat. TOKU	PARIS	JASS CLUB PARIS	2026-03-21 00:00	19:30, 21:30, 22:30	✔ oui	7
9	Noé Codjia Quartet 'Blue Codj' /	PARIS	JASS CLUB PARIS	2026-03-22 00:00	19:30	✔ oui	8
10	Deep Chuckles	PARIS	JASS CLUB PARIS	2026-03-25 00:00	19:30, 21:30	✔ oui	9
11	Jérémy Bruger Trio	PARIS	JASS CLUB PARIS	2026-03-26 00:00	19:30, 21:30	✔ oui	10
12	Apostolos Sideris Quintet	PARIS	JASS CLUB PARIS	2026-03-27 00:00	19:30, 21:30, 22:30	✔ oui	11

uv run python .\scripts\faiss_read_index.py

FAISS Vector Store

#	Titre	Ville	Vecteur [6 premières dims…]
0	Alexandre Herer 'Oxyd'	PARIS	[-0.0193, +0.0750, +0.0524, +0.0089, +0.0196, +0.0438 …]
1	Idrîs-Félix Bahri / Hommage à	PARIS	[-0.0460, +0.0315, +0.0347, +0.0011, +0.0269, +0.0489 …]
2	Neil Saidi & Yûr Yesli Quartet	PARIS	[-0.0101, +0.0521, +0.0411, +0.0230, +0.0252, +0.0466 …]
3	St Patrick / Entrée Libre ! Ja	PARIS	[-0.0185, +0.0217, +0.0503, +0.0209, +0.0293, +0.0361 …]
4	ZAMAKAN	PARIS	[-0.0089, +0.0278, +0.0471, -0.0015, +0.0359, +0.0632 …]
5	Azilis	PARIS	[-0.0284, +0.0469, +0.0251, +0.0088, -0.0001, +0.0268 …]
6	Virginie Daïdé Quartet 'While	PARIS	[-0.0099, +0.0344, +0.0239, +0.0159, +0.0095, +0.0464 …]
7	Paris Tokyo Exchange feat. TOK	PARIS	[-0.0111, +0.0356, +0.0509, -0.0110, +0.0406, +0.0353 …]
8	Noé Codjia Quartet 'Blue Codj'	PARIS	[-0.0175, +0.0383, +0.0370, +0.0090, +0.0287, +0.0262 …]
9	Deep Chuckles	PARIS	[-0.0432, +0.0453, +0.0152, +0.0030, +0.0153, +0.0320 …]
10	Jérémy Bruger Trio	PARIS	[-0.0128, +0.0273, +0.0580, +0.0041, +0.0140, +0.0572 …]
11	Apostolos Sideris Quintet	PARIS	[-0.0138, +0.0340, +0.0535, +0.0230, +0.0291, +0.0376 …]
12	Maë Defays 'A Deeper Ocean'	PARIS	[-0.0004, +0.0619, +0.0561, +0.0065, +0.0195, +0.0268 …]

3

Pipeline RAG Query

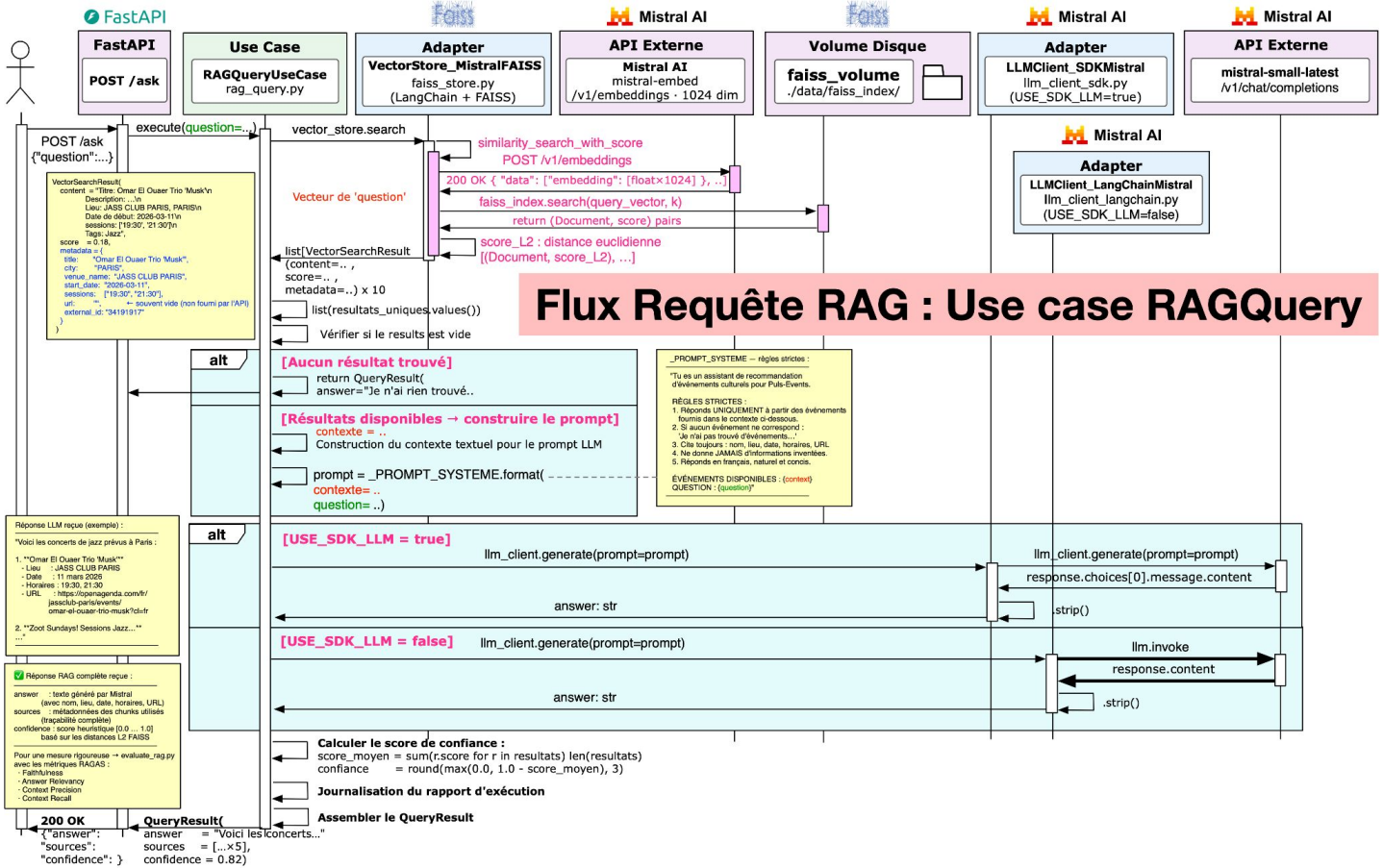
Flux Requête RAG

ÉTAPE 1 RECHERCHE VECTORIELLE
(IVectorStore + Mistral Embed)

ÉTAPE 2 CONSTRUCTION PROMPT
(RAGQueryUseCase)

ÉTAPE 3 GÉNÉRATION LLM
(ILLMClient → MistralLLMClient)

ÉTAPE 4 SCORE DE CONFIANCE
(heuristique L2)



3

Pipeline RAG Query

Pipeline RAG Query

Flux Requête RAG

ÉTAPE 1 RECHERCHE VECTORIELLE
(IVectorStore + Mistral Embed)

ÉTAPE 2 CONSTRUCTION PROMPT
(RAGQueryUseCase)

ÉTAPE 3 GÉNÉRATION LLM
(ILLMClient → MistralLLMClient)

ÉTAPE 4 SCORE DE CONFIANCE
(heuristique L2)

uv run python .\scripts\uc_RAGQueryUseCase.py

```
================================================================================
TEST RÉEL — RAGQueryUseCase.execute()
================================================================================
  Question : Quels concerts de jazz sont prévus à Paris ce mois-ci  ?
  Ville....: toutes
  Top-K....: 20
================================================================================
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
and faster downloads.

  Store vectoriel : Mistral cloud      (dim 1024)
  Index FAISS chargé..: 159 vecteurs
  Client LLM.........: SDK natif Mistral
WARNING:src.adapters.faiss.faiss_store:faiss.IDSelectorArray / SearchParameters non supporté par cette version de faiss — fallback mini-index
DEBUG: FAISS a retourné 23 résultats.
DEBUG: Apres filtrage par dates:  23 résultats.

================================================================================
RÉSULTAT — QueryResult retourné par RAGQueryUseCase.execute()
================================================================================
  Confiance..........: 0.571
  Sources utilisées...: 20

  Sources :
    [1] Omar El Ouaer Trio 'Musk'                          | 2026-03-11 00:00:00+00:00
    [2] Zoot Sundays! Sessions Jazz du dimanche            | 2026-03-01 00:00:00+00:00
    [3] Fred Nardin Trio                                   | 2026-03-05 00:00:00+00:00
….

  Réponse :
  --------------------------------------------------------------------------------
  Voici les concerts de jazz prévus à Paris en mars 2026 :

  1. **Omar El Ouaer Trio 'Musk'**
     - Lieu : JASS CLUB PARIS
     - Date : 11/03/2026 (19h30 & 21h30)
     - URL : [Lien](https://openagenda.com/fr/jassclub-paris/events/omar-el-ouaer-trio-musk?cl=fr) ….
```

4

Évaluation automatisée avec RAGAS

Mesure de Qualité RAG

Audit & Métriques RAGAS

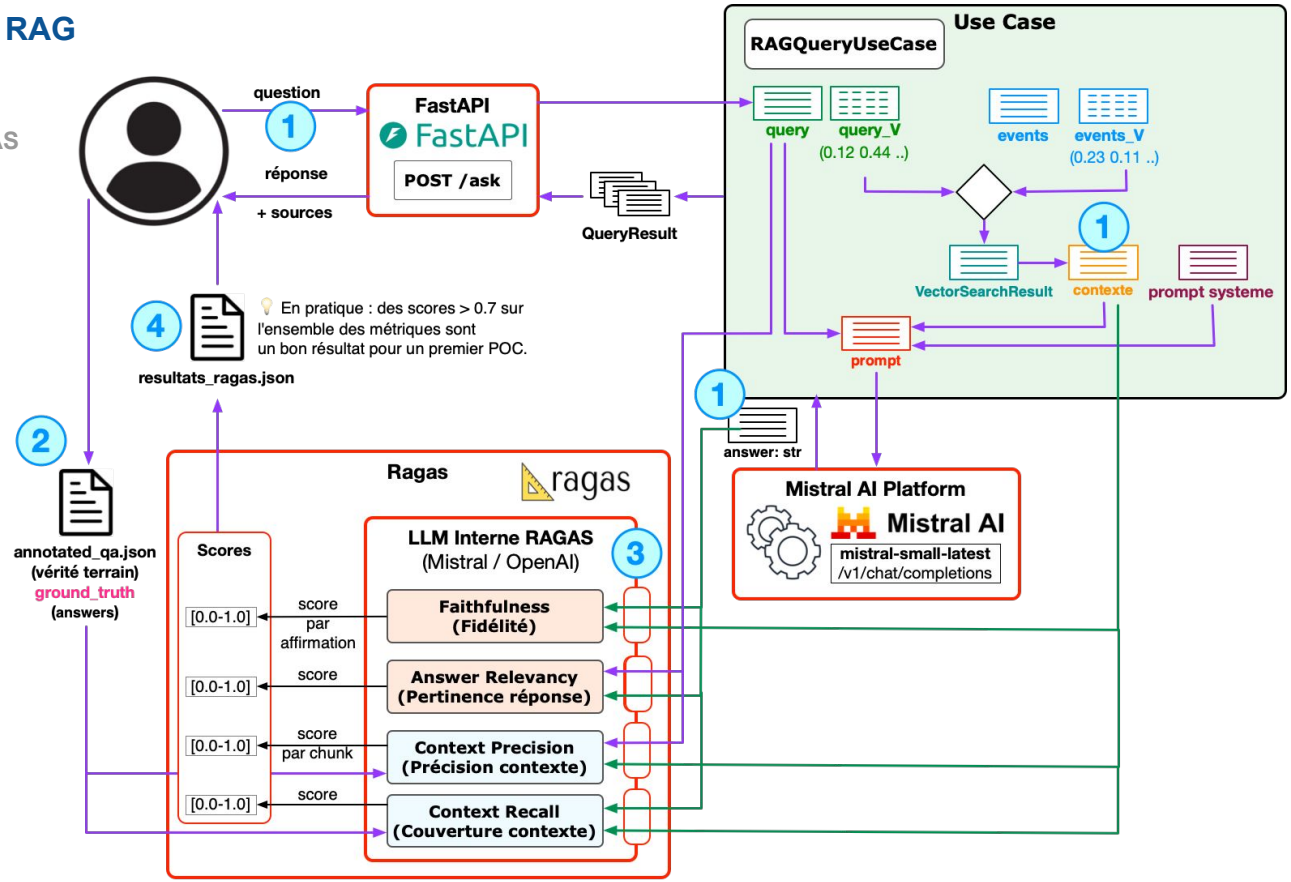
Le cycle d'amélioration itératif

1. Générer les réponses (answer) et contextes (contexts) avec le système RAG

2. Définir les ground_truth (réponses de référence)

3. Formater les données pour Ragas (avec datasets)

4. Enregistrer résultats.



4

Évaluation automatisée avec RAGAS

Mesure de Qualité RAG

Audit & Métriques RAGAS

Chargement du Dataset : Importation du fichier *annotated_qa.json* contenant les questions et les réponses attendues.

Inférence par API : Boucle de requêtes vers l'interface FastAPI pour obtenir les réponses du système RAG.

Calcul des Métriques : Analyse automatisée par RAGAS utilisant le modèle Mistral pour évaluer les métriques.

Analyse de Performance : Évaluation des métriques pour valider l'efficacité du moteur de recherche vectoriel.

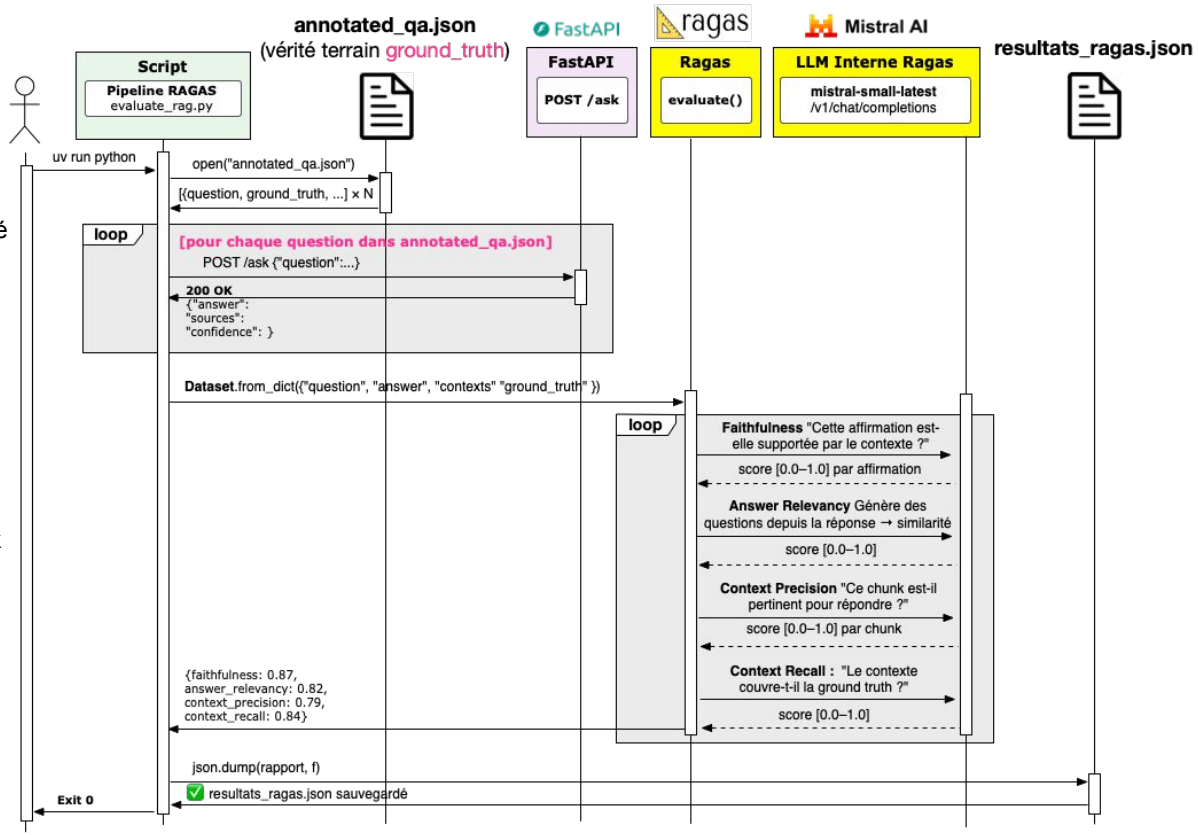
Export des Résultats : Génération du rapport final *resultats_ragas.json*

Boucle de Génération des Réponses (Inférence)

Le script parcourt le fichier de vérité terrain (`annotated_qa.json`) et envoie chaque question à l'API FastAPI.

Boucle de évaluation automatisée avec RAGAS

Soumettre les triplets [Question / Contexte / Réponse] au framework RAGAS pour une évaluation mathématique et qualitative.



4

Évaluation automatisée avec RAGAS

Mesure de Qualité RAG

Étapes de l'Évaluation

Chargement du Dataset : Importation du fichier *annotated_qa.json* contenant les questions et les réponses attendues.

Inférence par API : Boucle de requêtes vers l'interface FastAPI pour obtenir les réponses du système RAG.

Calcul des Métriques : Analyse automatisée par RAGAS utilisant le modèle Mistral pour évaluer les métriques.

Analyse de Performance : Évaluation des métriques pour valider l'efficacité du moteur de recherche vectoriel.

Export des Résultats : Génération du rapport final *resultats_ragas.json*

1er Boucle de Génération des Réponses (Inférence)

```
--------------------------------------------------------------------------------
LOOP [1/14]
  Question.....................: Quels concerts jazz sont prévus à Paris ?
  Réponse......................: Voici les concerts jazz prévus à Paris :

1. **Zoot Sun
  Score FAISS (Distancia L2)...: 0.3784
  Confianza RAG (Probabilidad).: 0.6180
  * Similarité cosinus.........: 0.1322
  * Couverture clés............: 1.0000
  * Classification.............: correcte

--------------------------------------------------------------------------------
LOOP [2/14]
  Question.....................: Y a-t-il un concert de Mourad Benhammou à Paris en nove...
  Réponse......................: Oui, il y a un concert de Mourad Benhammou à Paris en n...
  Score FAISS (Distancia L2)...: 0.3127
  Confianza RAG (Probabilidad).: 0.5680
  * Similarité cosinus.........: 0.0677
  * Couverture clés............: 1.0000
  * Classification.............: correcte
```

2de Boucle d'évaluation automatisée avec RAGAS

```
Evaluating: 100%|████████████████████████████████████████████████| 42/42 [02:31<00:00,  3.60s/it]
2026-03-13 16:33:21,337 | WARNING | RAG-Eval | Fallback extraction scores (KeyError sur dict(resultat))

================================================================================
  RÉSULTATS RAGAS — Puls-Events RAG
================================================================================
  Faithfulness........: 0.4114  (cible > 0.85)
  Context Precision...: 0.4286  (cible > 0.75)
  Context Recall......: 0.2143  (cible > 0.8)
  Answer Relevancy....: N/A  (exclu — bug LangchainEmbeddingsWrapper)
================================================================================

  ⚠️  ATTENTION — certains seuils non atteints
================================================================================
```

4

Évaluation automatisée avec RAGAS

Chargement du Dataset : Importation du fichier *annotated_qa.json* contenant les questions et les réponses attendues.

Inférence par API : Boucle de requêtes vers l'interface FastAPI pour obtenir les réponses du système RAG.

Calcul des Métriques : Analyse automatisée par RAGAS utilisant le modèle Mistral pour évaluer les métriques.

Analyse de Performance : Évaluation des métriques pour valider l'efficacité du moteur de recherche vectoriel.

Export des Résultats : Génération du rapport final *resultats_ragas.json*

Mesure de Qualité RAG

Étapes de l'Évaluation

Boucle de évaluation automatisée avec RAGAS V2

```
================================================================================
  RAPPORT D'ÉVALUATION RAGAS — Puls-Events RAG
================================================================================
  Métrique                        Score      Seuil   Statut
  ──────────────────────────────────────────────────────────
  Faithfulness                   0.6989       0.85   ⚠️   (pas d'hallucinations)
  Context Precision              0.7024       0.75   ⚠️   (qualité des chunks FAISS)
  Context Recall                 1.0000       0.80   ✅  (couverture de la vérité terrain)
  Answer Relevancy                  N/A          —    (exclu — bug LangchainEmbeddingsWrapper)
  ──────────────────────────────────────────────────────────
```

- **Ajout de Few-Shot Prompting**
- **Optimisation du Prompt Système**
- **Optimisation du GroundTruth .**

Analyse des Métriques RAGAS

- **Context Recall (1.00) :** Les chunks récupérés via **FAISS** contiennent l'intégralité des informations nécessaires pour répondre. L'étape de *retrieval* est parfaitement fonctionnelle.
- **Context Precision (0.70) :** Indique qu'environ 30 % des chunks récupérés sont moins pertinents que les autres.
- **Faithfulness (0.70) :** Ce score s'explique techniquement par le fait que le LLM ajoute occasionnellement des éléments de mise en forme ou de contextualisation (ex: *"Tous les dimanches à partir de..."*) qui dépassent légèrement le contenu strict du chunk. Il ne s'agit pas d'hallucinations critiques, mais ***d'enrichissements mineurs***.

Pistes d'amélioration identifiées :

- **Context Precision** : Implémenter un re-ranking (MMR ou cross-encoder) pour filtrer les chunks peu pertinents avant l'injection.
- **Context Precision** : Réduire k (de 5 à 3 chunks) pour forcer une sélection plus resserrée.
- **Faithfulness**: Renforcer le "System Prompt" avec une règle stricte : *"Ne cite que les informations explicitement présentes dans le contexte".*
- ***Classifieur d'intention*** *(RAG vs CHAT)*

5

Docker
& Démo

Conteneurisation & API

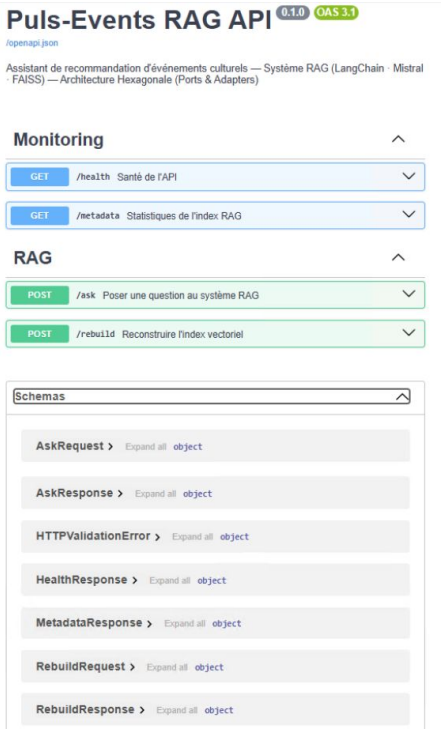
Déploiement RAG

Mise en Production avec Docker Compose

docker compose --env-file .env -f docker/docker-compose.yml up -d api

- **Dockerisation de l'API :** Création du Dockerfile optimisé avec l'installateur uv et Python 3.12-slim.
- **Orchestration multi-services :** Configuration du docker-compose.yml pour lier l'API, la base de données PostgreSQL et l'interface pgAdmin.
- **Gestion de l'Index Vectoriel :** Montage de volumes pour la persistance de l'index FAISS et des données traitées.
- **Exposition du service :** Déploiement du serveur Uvicorn et validation des endpoints via l'interface Swagger/OpenAPI.

```
#1 [internal] load local bake definitions
#2 [internal] load build definition from Dockerfile
#3 [internal] load metadata for docker.io/library/python:3.12-slim
#4 [internal] load .dockerignore
#5 [1/7] FROM docker.io/library/python:3.12-slim@sha256….
#6 [internal] load build context
#7 [2/7] RUN apt-get update && apt-get install -y ….
#8 [3/7] RUN pip install --no-cache-dir uv
#9 [4/7] WORKDIR /app
#10 [5/7] COPY requirements.txt ./
#11 [6/7] RUN uv pip install --system --no-cache -r requirements.txt
#12 [7/7] COPY . .
#13 exporting to image
#14 resolving provenance for metadata file
#14 DONE 0.1s
[+] up 3/3
 ✔ Image puls-events-rag-api Built
1948.8s
 ✔ Container puls_events_db  Healthy
7.1s
 ✔ Container puls_events_api Created
```




```
PS C:\Users\Public\IAE_DELL\pra_dell\m7_ocr> docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS                            PORTS                    NAMES
ab8e38ce90b0   puls-events-rag-api    "uvicorn src.api.mai…"   5 minutes ago    Up 4 seconds (health: starting)   0.0.0.0:8007->8000/tcp   puls_events_api
734412fdc003   postgres:15            "docker-entrypoint.s…"   10 days ago      Up 5 days (healthy)               0.0.0.0:5437->5432/tcp   puls_events_db
6f51f11ee0c7   dpage/pgadmin4:latest  "/entrypoint.sh"         10 days ago      Up 4 days                         0.0.0.0:8087->80/tcp     puls_events_pgadmin
```

Q & A